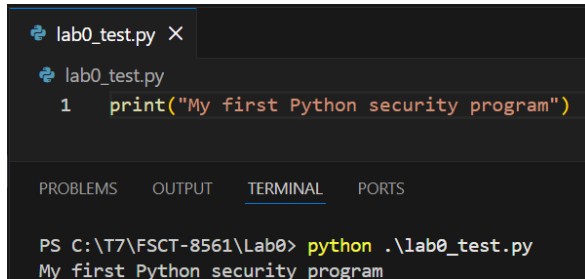


Lab 0 - Introduction to Network Protocols Using Python

Sockets

Part 1 — Your First Python Program

Run the file

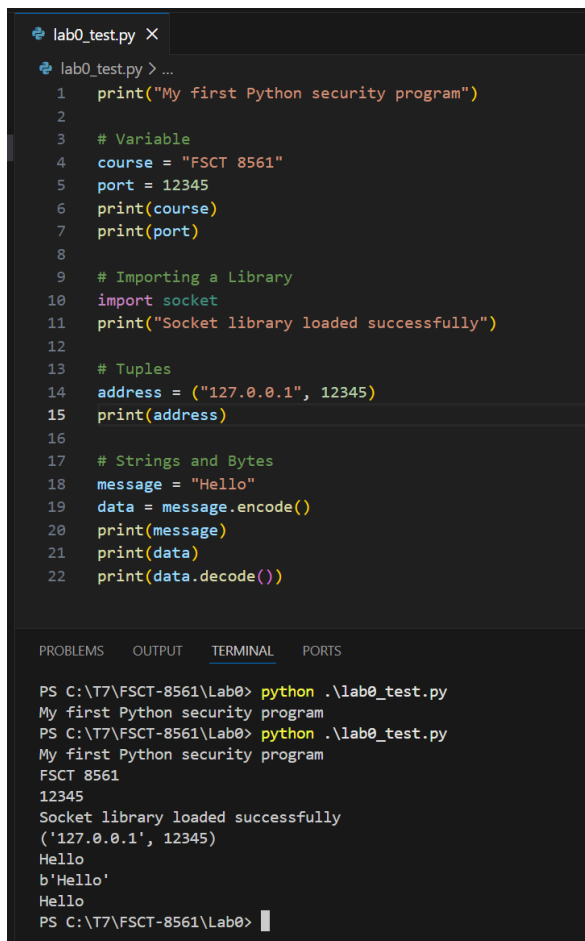


```
lab0_test.py X
lab0_test.py
1 print("My first Python security program")

PROBLEMS OUTPUT TERMINAL PORTS

PS C:\T7\FSC-8561\Lab0> python .\lab0_test.py
My first Python security program
```

Minimum Python Basics for This Lab



```
lab0_test.py X
lab0_test.py > ...
1 print("My first Python security program")
2
3 # Variable
4 course = "FSC 8561"
5 port = 12345
6 print(course)
7 print(port)
8
9 # Importing a Library
10 import socket
11 print("Socket library loaded successfully")
12
13 # Tuples
14 address = ("127.0.0.1", 12345)
15 print(address)
16
17 # Strings and Bytes
18 message = "Hello"
19 data = message.encode()
20 print(message)
21 print(data)
22 print(data.decode())

PROBLEMS OUTPUT TERMINAL PORTS

PS C:\T7\FSC-8561\Lab0> python .\lab0_test.py
My first Python security program
PS C:\T7\FSC-8561\Lab0> python .\lab0_test.py
My first Python security program
FSC 8561
12345
Socket library loaded successfully
('127.0.0.1', 12345)
Hello
b'Hello'
Hello
PS C:\T7\FSC-8561\Lab0> 
```

Part 2 — Understanding Sockets

```
Labs > lab0_part2.py > ...
1  import socket
2  my_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
3  print("TCP socket created successfully")
4  my_socket.close()
```

PROBLEMS OUTPUT TERMINAL PORTS

```
PS C:\T7\FSC-8561\Lab0\Labs> py .\lab0_part2.py
TCP socket created successfully
PS C:\T7\FSC-8561\Lab0\Labs> 
```

Reflection Question 1

1. What is a socket?

A socket is an interface that allows an application to send and receive data over a network. It is uniquely identified by combining an IP address and a port number.

2. What does `AF_INET` mean?

It means that the socket will use **IPv4** addresses.

3. What does `SOCK_STREAM` mean?

It means that the socket will use **TCP** protocol.

Part 3 — Client-Server Communication

Part 4 — Create a Basic TCP Server

Run the Server

```
Labs > server.py > ...
1  import socket
2
3  HOST = "127.0.0.1"
4  PORT = 12345
5
6  server_socket = socket.socket(
7      socket.AF_INET,
8      socket.SOCK_STREAM
9  )
10
11 server_socket.bind((HOST, PORT))
12 server_socket.listen(1)
13
14 print("Server is waiting for a connection...")
15
16 client_socket, client_address = server_socket.accept()
17
18 print("Connected by:", client_address)
19
20 data = client_socket.recv(1024)
21
22 message = data.decode()
23
24 print("Client says:", message)
25
26 reply = "Message received"
27
28 client_socket.send(reply.encode())
29
30 client_socket.close()
31 server_socket.close()
```

PROBLEMS OUTPUT TERMINAL PORTS

```
PS C:\T7\FSCT-8561\Lab0\Labs> python .\server.py
Server is waiting for a connection...
```

Part 5 — Create a Basic TCP Client

Run the Client

Labs >  client.py > ...

```
1  import socket
2
3  HOST = "127.0.0.1"
4  PORT = 12345
5
6  client_socket = socket.socket(
7      socket.AF_INET,
8      socket.SOCK_STREAM
9  )
10 (variable) client_socket: socket
11 client_socket.connect((HOST, PORT))
12
13 message = "Hello Server"
14
15 client_socket.send(message.encode())
16
17 response = client_socket.recv(1024)
18
19 print("Server replied:", response.decode())
20
21 client_socket.close()
```

PROBLEMS OUTPUT TERMINAL PORTS

```
PS C:\T7\FSCT-8561\Lab0\Labs> python .\client.py
Server replied: Message received
PS C:\T7\FSCT-8561\Lab0\Labs> 
```

```
PS C:\T7\FSCT-8561\Lab0\Labs> python .\server.py
Server is waiting for a connection...
Connected by: ('127.0.0.1', 25923)
Client says: Hello Server
PS C:\T7\FSCT-8561\Lab0\Labs> 
```

Part 6 — Modify the Client and Server

Task A - Change the client message to include your name.

```
PS C:\T7\FSCT-8561\Lab0\Labs> python .\server.py
Server is waiting for a connection...
Connected by: ('127.0.0.1', 62024)
Client says: Hello Server, this is Kuangda Zhu.
PS C:\T7\FSCT-8561\Lab0\Labs>
```

Task B - Change the server reply.

```
PS C:\T7\FSCT-8561\Lab0\Labs> python .\client.py
Server replied: This is the Server, everything is OK.
PS C:\T7\FSCT-8561\Lab0\Labs>
```

Task C — Small Challenge

```
26     reply = "Server received: " + message
27
```

PROBLEMS OUTPUT TERMINAL PORTS

```
PS C:\T7\FSCT-8561\Lab0\Labs> python .\client.py
Server replied: Server received: Hello Server, this is Kuangda Zhu.
PS C:\T7\FSCT-8561\Lab0\Labs>
```

Part 7 — Connect to a Public Web Server

```
PS C:\T7\FSCT-8561\Lab0\Labs> python .\http_client.py
HTTP/1.1 200 OK
Date: Thu, 10 Sep 2026 18:12:37 GMT
Content-Type: text/html
Transfer-Encoding: chunked
Connection: close
Server: cloudflare
Last-Modified: Sun, 30 Aug 2026 04:11:49 GMT
Allow: GET, HEAD
Accept-Ranges: bytes
Age: 3615
cf-cache-status: HIT
CF-RAY: a39066e94c642c2e-YVR

22f
<!doctype html><html lang="en"><head><title>Example Domain</title><link rel="icon" href="data:,"><meta name="viewport" content="width=device-width, initial-scale=1"><style>body{background:#eee;width:60vw;margin:15vh auto;font-family:system-ui,sans-serif}h1{font-size:1.5em}div{opacity:0.8}a:link,a:visited{color:#348}</style></head><body><div><h1>Example Domain</h1><p>This domain is for use in documentation examples without needing permission. Avoid use in operations.</p><p><a href="https://iana.org/domains/example">Learn more</a></p></div></body></html>

0
```

Reflection Question 2

1. Which host did the client connect to?

example.com

2. Which port was used?

3. Why is port 80 relevant?

Because port 80 is the standard port used for HTTP.

4. What did the server send back?

HTTP Response Headers and HTTP Response Body (HTML).

Part 8 - Challenge — Rebuild the Client-Server Application Using Socket.IO

The Socket.IO server running,

```
1  from aiohttp import web
2  import socketio
3
4  socket_io = socketio.AsyncServer()
5
6  app = web.Application()
7  socket_io.attach(app)
8
9  async def index(request):
10     return web.Response(text='Hello world this is a socket server.', content_type='text/html')
11
12 @socket_io.on('message')
13 def print_message(socket_id, data):
14     print("Socket ID: ", socket_id)
15     print("Data: ", data)
16
17 app.router.add_get('/', index)
18
19 if __name__ == '__main__':
20     web.run_app(app)
```

PROBLEMS OUTPUT TERMINAL PORTS

```
PS C:\T7\FSCT-8561\Lab0\Labs> python .\server_socket_io.py
===== Running on http://0.0.0.0:8080 =====
(Press CTRL+C to quit)
```

The client connecting,

```
PROBLEMS OUTPUT TERMINAL PORTS
```

```
PS C:\T7\FSCT-8561\Lab0\Labs> python .\client_socket_io.py
websocket-client package not installed, only polling transport is available
connection established
Enter to dc...█
```

The message received by the server,

```
PROBLEMS  OUTPUT  TERMINAL  PORTS

PS C:\T7\FSCIT-8561\Lab0\Labs> python .\server_socket_io.py
===== Running on http://0.0.0.0:8080 =====
(Press CTRL+C to quit)
Socket ID:  j1Cfzh_H9n1nkeHWAAB
Data:  I am Kuangda, I am sad.
█
```

The client disconnecting.

```
PS C:\T7\FSCIT-8561\Lab0\Labs> python .\client_socket_io.py
websocket-client package not installed, only polling transport is available
connection established
Enter to dc...
disconnected from server
PS C:\T7\FSCIT-8561\Lab0\Labs> █
```

Part 8 — Security Reflection

1. What could happen if a server accepts connections from anyone?

It exposes the server to unauthorized access which makes it unsafe.

2. Is the Lab 0 message encrypted?

No.

3. Could network traffic be observed by another system?

Yes, it can be captured and read using tools like Wireshark.

4. Why should servers validate input?

To prevent injection attacks and malicious code execution

5. Why might authentication be useful?

It verifies client's identities and ensures that only permitted users can access the server.

6. Suggest two ways to make this application more secure.

1. Message encryption
2. User authentication

Part 9 — Deliverables

A short paragraph explaining what you learned and what was unclear

In Lab 0, I learned how to build basic client-server connections using TCP and Socket.IO in Python, while gaining hands-on experience with socket programming. Now I have a solid understanding of point-to-point TCP connections; however, it is still unclear to me how multi-to-multi connections work using UDP.

Github Link

<https://github.com/bnwzc/FSCT8561-Labs-Kuangda-Zhu>